

SE/COM S 3190 – Construction of User Interfaces  
Final Project Technical Report

Spring 2026

# Ames Housing Finder

Team AL8

Vikrant Gandotra      Madhava Nallamilli  
gandotra@iastate.edu   mnalla23@iastate.edu

*Iowa State University*  
*Department of Computer Science*

# Contents

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                            | <b>3</b>  |
| 1.1      | Project Overview . . . . .                     | 3         |
| 1.2      | Target Users and Use Cases . . . . .           | 3         |
| 1.3      | Core Features . . . . .                        | 3         |
| 1.4      | Originality vs. Inspiration . . . . .          | 4         |
| <b>2</b> | <b>System Overview and Project Description</b> | <b>5</b>  |
| 2.1      | Major Functional Modules . . . . .             | 5         |
| 2.2      | UI and Navigation Flow . . . . .               | 5         |
| 2.3      | CRUD Entities . . . . .                        | 6         |
| <b>3</b> | <b>File and Folder Architecture</b>            | <b>7</b>  |
| 3.1      | Folder Structure Overview . . . . .            | 7         |
| 3.2      | Backend Folder Structure . . . . .             | 7         |
| 3.2.1    | Backend Folder Tree . . . . .                  | 7         |
| 3.2.2    | Backend Folder Description . . . . .           | 8         |
| 3.3      | Frontend Folder Structure . . . . .            | 8         |
| 3.3.1    | Frontend Folder Tree . . . . .                 | 8         |
| 3.3.2    | Frontend Folder Description . . . . .          | 9         |
| <b>4</b> | <b>Code Explanation and Logic Flow</b>         | <b>10</b> |
| 4.1      | Frontend–Backend Interaction . . . . .         | 10        |
| 4.2      | React Component Example . . . . .              | 10        |
| 4.3      | API Route Examples . . . . .                   | 11        |
| 4.4      | Database Interaction . . . . .                 | 12        |
| <b>5</b> | <b>Screenshots and Web Views</b>               | <b>13</b> |
| <b>6</b> | <b>Installation and Setup Instructions</b>     | <b>24</b> |
| 6.1      | Frontend Setup . . . . .                       | 24        |
| 6.2      | Backend Setup . . . . .                        | 24        |
| 6.3      | Database Setup . . . . .                       | 25        |
| 6.4      | Environment Variables . . . . .                | 25        |
| <b>7</b> | <b>Contribution Overview</b>                   | <b>26</b> |
| 7.1      | Vikrant Gandotra Contributions . . . . .       | 26        |
| 7.2      | Madhava Nallamilli Contributions . . . . .     | 26        |
| <b>8</b> | <b>Challenges Faced</b>                        | <b>27</b> |

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| 8.1      | Vikrant Gandotra Challenges . . . . .   | 27        |
| 8.2      | Madhava Nallamilli Challenges . . . . . | 28        |
| <b>9</b> | <b>Final Reflections</b>                | <b>28</b> |
| 9.1      | Vikrant Gandotra Reflection . . . . .   | 28        |
| 9.2      | Madhava Nallamilli Reflection . . . . . | 29        |

# 1 Introduction

## 1.1 Project Overview

Ames Housing Finder is a full-stack web application for students searching for housing near Iowa State University. The problem the project addresses is that students often need to compare rent, location, property type, availability, reviews, and tour options across multiple apartments or houses. When this information is scattered across different websites, spreadsheets, messages, or notes, the search process becomes harder to manage.

The application provides a centralized interface where users can browse listings, view listing details, save favorites, compare choices, schedule housing tours, leave reviews, and read housing resources. The project is implemented as a React single page application connected to a Node.js/Express backend and a MongoDB database. The final product is an interactive housing search and management system with guest, student, and administrator workflows.

## 1.2 Target Users and Use Cases

The primary users are Iowa State students who are looking for housing in Ames. A student can browse available listings, save promising options, compare up to three listings, request a tour, and write reviews about housing experiences. These workflows support the realistic process of narrowing housing choices before signing a lease.

Secondary users are administrators who manage the content of the application. Admins can create and maintain listing data, manage housing resources, moderate reviews by deleting inappropriate content, and update tour requests. Guests can still browse public pages such as Home, Listings, Listing Details, and Info/FAQ before creating an account.

## 1.3 Core Features

The main capabilities of the system are:

- **Listings and listing details:** users can browse database-backed housing listings and open a detailed page for each property.
- **Search, filtering, and sorting:** listings can be filtered by text, rent range, bedroom

count, property type, status, and sort order.

- **Authentication:** users can sign up as students, log in, remain signed in through local storage, and log out.
- **Role-based behavior:** guests have public access, students can use personal features, and admins can manage protected data.
- **Favorites CRUD:** signed-in users can save listings, add labels or notes, prioritize options, update records, and delete saved records.
- **Compare CRUD:** users can add up to three listings to a compare list, update comparison notes, and remove selections.
- **Tour Appointments CRUD:** users can request tours, edit tour information, confirm or cancel tours, and delete appointment records.
- **Reviews:** students can create reviews; users can edit or delete their own reviews; admins can delete reviews for moderation but cannot edit another user's review.
- **Resources/FAQ:** the application stores housing information resources and FAQ entries in MongoDB, with admin creation, editing, deletion, and category filtering.
- **Dashboard:** signed-in users get a summary of listing counts, favorites, compare records, tours, reviews, and resources.

## 1.4 Originality vs. Inspiration

The project is original in implementation and data model, but the idea is inspired by real housing search websites and student review platforms. Common sites allow people to browse apartments and save favorites, while review platforms allow students to share experiences. Ames Housing Finder combines these ideas into one student-centered application for Iowa State housing. The difference is that the application emphasizes course-project requirements such as clearly separated CRUD modules, role behavior, a database-backed FAQ/resources page, dashboard summaries, and a simple interface designed specifically for Ames student housing.

## 2 System Overview and Project Description

### 2.1 Major Functional Modules

The application is divided into a frontend React client and a backend Express API.

- **Frontend routing module:** `frontend/src/App.jsx` uses React Router to define public pages, protected pages, and the 404 route.
- **Navigation and layout module:** `AppNavbar.jsx`, `Footer.jsx`, and `index.css` provide consistent page structure and styling.
- **Listings module:** listing pages and listing routes provide search, filtering, sorting, detail viewing, and admin listing management.
- **Personal saved data modules:** favorites and compare records are tied to the current user's id and stored in separate MongoDB collections.
- **Appointments module:** tour requests connect a user, listing, visitor name, visitor email, date, status, and note.
- **Reviews module:** reviews connect a user and listing with rating, title, author name, and comment.
- **Resources/FAQ module:** resources support categories such as FAQ, Budget, Lease, Moving, and Campus; featured resources appear on the dashboard and home page.
- **Authentication module:** signup and login routes validate users against MongoDB and return a user object used by the frontend session service.
- **Dashboard module:** the dashboard route aggregates counts and recent records for a signed-in user's account page.

### 2.2 UI and Navigation Flow

A guest begins at the Home page and can browse public pages such as Listings, Listing Details, and Info/FAQ. The navigation bar shows Login and Sign Up when no user is authenticated. After login, the navigation changes to show the signed-in user's name and protected links such as Dashboard, Favorites, Compare, Tours, Reviews, and Resources.

A typical student workflow is: browse listings, open a listing detail page, save it to favorites, add it to compare, request a tour, then optionally write a review. A typical admin workflow is: log in with the seeded admin account, create or update listing/resource data, review dashboard summaries, update tour statuses, and delete inappropriate reviews. Invalid routes are caught by the 404 page and return the user to Home.

## 2.3 CRUD Entities

| Entity           | Important Fields                                                                                                                  | Supported Actions                                                                                                                                                  |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Listing          | title, address, neighborhood, rent, bedrooms, bathrooms, squareFeet, propertyType, availableDate, amenities, contactEmail, status | Read by all users; create, update, and delete by admins. Listings are also referenced by favorites, compare records, tours, and reviews.                           |
| Favorite         | userId, listingId, label, priority, note                                                                                          | Create, read, update, and delete personal saved listings. Unique index prevents duplicate favorite records for the same user and listing.                          |
| CompareSelection | userId, listingId, compareNote                                                                                                    | Create, read, update, and delete comparison records. Unique index prevents duplicate compare records; backend limits each user to three active compare selections. |
| TourAppointment  | userId, listingId, visitorName, visitorEmail, tourDate, status, note                                                              | Create, read, update, confirm/-cancel, and delete tour requests. Status is constrained to Requested, Confirmed, Completed, or Canceled.                            |
| ListingReview    | userId, listingId, authorName, rating, title, comment                                                                             | Create and read reviews. Users can edit/delete their own reviews; admins can delete reviews but cannot edit reviews written by others.                             |

| Entity          | Important Fields                        | Supported Actions                                                                                                   |
|-----------------|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| HousingResource | category, title, body, link, isFeatured | Create, read, update, delete, filter by category, and display featured items on the dashboard/home page.            |
| User            | name, email, passwordHash, role         | Signup creates Student users; login retrieves matching user records; seeded admin user enables admin-only behavior. |

## 3 File and Folder Architecture

### 3.1 Folder Structure Overview

The project uses separate `frontend/` and `backend/` directories. This split keeps React UI code separate from Express API code, which makes the development workflow easier: the frontend can run through Vite on one port while the backend API runs on port 5000. The `Documents/` folder stores project documentation, demo checklist material, and video script drafts.

### 3.2 Backend Folder Structure

#### 3.2.1 Backend Folder Tree

```
backend/
|-- .env.example
|-- package.json
|-- server.js
|-- config/
|   |-- db.js
|-- api/
|   |-- seed.js
|   |-- models/
|       |-- CompareSelection.js
```

```

|   |   |-- Favorite.js
|   |   |-- HousingResource.js
|   |   |-- Listing.js
|   |   |-- ListingReview.js
|   |   |-- TourAppointment.js
|   |   |-- User.js
|   |-- routes/
|       |-- appointments.js
|       |-- auth.js
|       |-- compare.js
|       |-- dashboard.js
|       |-- favorites.js
|       |-- listings.js
|       |-- resources.js
|       |-- reviews.js

```

### 3.2.2 Backend Folder Description

`backend/server.js` creates the Express application, enables CORS and JSON parsing, connects to MongoDB, mounts all route files under `/api`, defines a health endpoint, and provides a fallback JSON 404 response. The `config/db.js` file isolates the Mongoose connection logic. The `api/models/` directory contains Mongoose schemas for each collection. The `api/routes/` directory contains Express routers that implement endpoint logic, validation, permissions, database reads/writes, and error responses. The `api/seed.js` script clears and repopulates the database with demo users, listings, favorites, compare records, tours, reviews, and resources.

## 3.3 Frontend Folder Structure

### 3.3.1 Frontend Folder Tree

```

frontend/
|-- package.json
|-- vite.config.js
|-- index.html
|-- src/

```

```
| |-- main.jsx
| |-- App.jsx
| |-- index.css
| |-- components/
| | |-- AppNavbar.jsx
| | |-- Footer.jsx
| | |-- ListingCard.jsx
| |-- pages/
| | |-- Appointments.jsx
| | |-- Compare.jsx
| | |-- Dashboard.jsx
| | |-- FAQ.jsx
| | |-- Favorites.jsx
| | |-- Home.jsx
| | |-- ListingDetails.jsx
| | |-- Listings.jsx
| | |-- Login.jsx
| | |-- NotFound.jsx
| | |-- Resources.jsx
| | |-- Reviews.jsx
| | |-- Signup.jsx
| |-- services/
| | |-- api.js
| | |-- session.js
```

### 3.3.2 Frontend Folder Description

The frontend is organized around pages and reusable components. The `pages/` folder contains route-level components that correspond to the navigation items. The `components/` folder stores shared UI pieces such as the navigation bar, footer, and listing cards. The `services/api.js` file centralizes Axios configuration with the backend base URL. The `services/session.js` file handles local storage, current user normalization, role helpers, and request payload helpers such as `userPayload()` and `actorQuery()`.

## 4 Code Explanation and Logic Flow

### 4.1 Frontend–Backend Interaction

The React frontend sends HTTP requests with Axios. The shared Axios instance is defined in `frontend/src/services/api.js`:

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:5000/api"
});

export default api;
```

A page component usually stores form data and loaded records in React state. When a user submits a form, the component validates required fields, sends a JSON body to the backend, updates state from the response, and displays a message. For protected records, the frontend sends `userId` and `actorId` values from `session.js`. The backend uses those values to check whether the user is logged in, whether the actor is an admin, and whether the record belongs to the current user.

### 4.2 React Component Example

The Compare page is a representative CRUD component. It loads listings and existing compare selections with `useEffect`, stores form fields with `useState`, posts a new compare selection, updates the compare note through PUT, and deletes selections through DELETE.

```
async function submitSelection(event) {
  event.preventDefault();

  if (!currentUser?._id) {
    setMessage("Please log in to compare listings.");
    return;
  }

  if (!form.listingId) {
```

```

    setMessage("Choose a listing before adding it to compare.");
    return;
}

if (editing) {
    await api.put(`/compare/${editing._id}`, {
        ...userPayload(currentUser),
        compareNote: form.compareNote
    });
    setMessage("Compare note updated.");
} else {
    await api.post("/compare", {
        ...userPayload(currentUser),
        listingId: form.listingId,
        compareNote: form.compareNote
    });
    setMessage("Listing added to compare.");
}

resetForm();
await loadData();
}

```

This pattern appears throughout the application: maintain local state, call the backend, handle the response, refresh the displayed records, and show an informational message when an operation succeeds or fails.

### 4.3 API Route Examples

The backend uses Express routers for each domain area. The main server mounts route modules under the `/api` prefix:

```

app.use("/api/auth", authRoutes);
app.use("/api/listings", listingsRoutes);
app.use("/api/favorites", favoritesRoutes);
app.use("/api/compare", compareRoutes);
app.use("/api/appointments", appointmentsRoutes);
app.use("/api/reviews", reviewsRoutes);

```

```
app.use("/api/resources", resourcesRoutes);
app.use("/api/dashboard", dashboardRoutes);
```

The following route examples show the CRUD pattern used across the project.

| HTTP Method | Endpoint              | Example | Behavior                                                                                                                                                                                                               |
|-------------|-----------------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET         | /api/listings         |         | Builds a MongoDB query from search, minRent, maxRent, bedrooms, propertyType, status, and sort query parameters. Returns matching listings sorted by rent, newest, bedrooms, or default createdAt order.               |
| POST        | /api/compare          |         | Requires a logged-in actor, validates listing id, verifies the listing exists, prevents duplicates, enforces a maximum of three compare records, creates a CompareSelection document, and returns the hydrated record. |
| PUT         | /api/appointments/:id |         | Loads the tour appointment, verifies the actor can update the record, updates date/status/note fields, saves with validators, and returns the updated appointment.                                                     |
| DELETE      | /api/reviews/:id      |         | Loads the review, checks whether the actor owns it or is an admin, prevents unauthorized edits/deletes, deletes the review document, and returns a confirmation message.                                               |

## 4.4 Database Interaction

The backend uses Mongoose models to define collections, validation, references, timestamps, and indexes. For example, `CompareSelection.js` references listings and prevents a user from adding the same listing twice:

```
const compareSelectionSchema = new mongoose.Schema(
  {
    userId: { type: String, required: true, trim: true },
    listingId: {
```

```

    type: mongoose.Schema.Types.ObjectId,
    ref: "Listing",
    required: true
  },
  compareNote: { type: String, default: "", trim: true }
},
{ timestamps: true }
);

compareSelectionSchema.index({ userId: 1, listingId: 1 }, { unique: true });

```

The Listing model includes searchable fields and a text index:

```

listingSchema.index({ title: "text", address: "text", neighborhood: "text" });

```

Most relationships are stored by `ObjectId` references, such as favorites, compare selections, appointments, and reviews referencing `Listing`. User ownership is stored as a string `userId` so records can be filtered by the active account. Common queries include `find()`, `findById()`, `findByIdAndUpdate()`, `findByIdAndDelete()`, `countDocuments()`, and `populate()` or custom hydration for listing references.

## 5 Screenshots and Web Views

This section shows the major pages of the application and explains the purpose of each page.

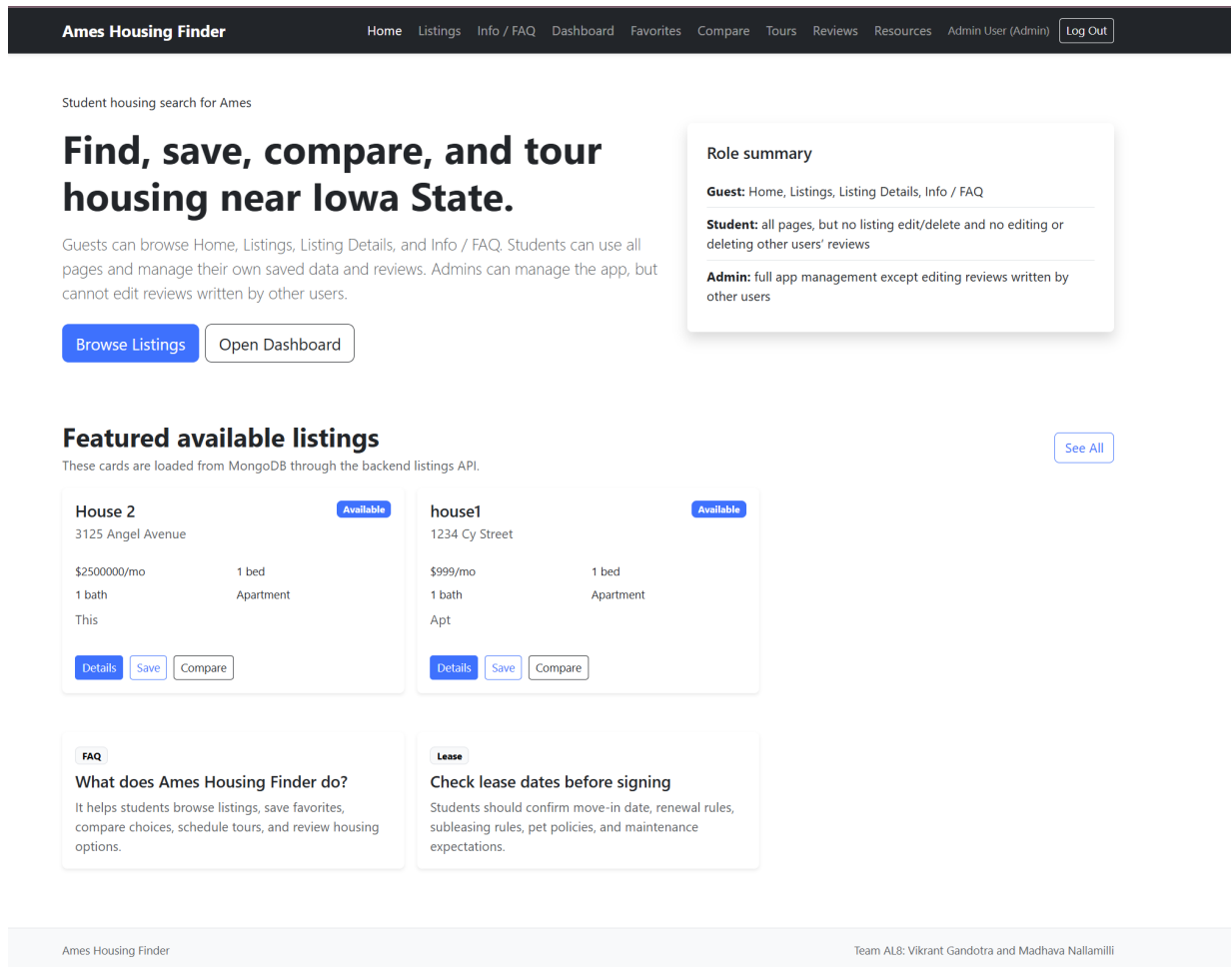


Figure 1: Home page with role summary, featured listings, and featured resources. Users can browse listings or open the dashboard after login.

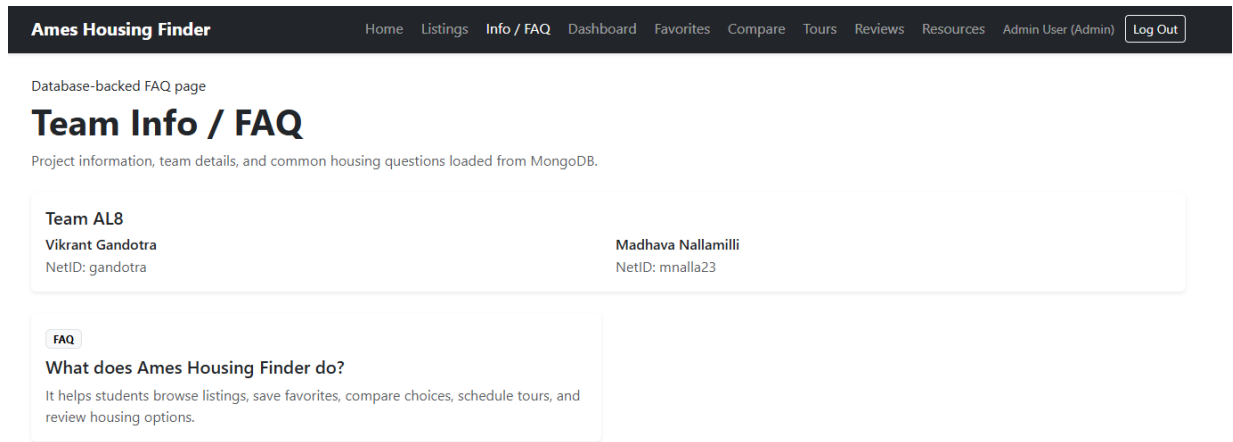


Figure 2: Info/FAQ page with team information and database-backed FAQ content. This page is public for guest users.

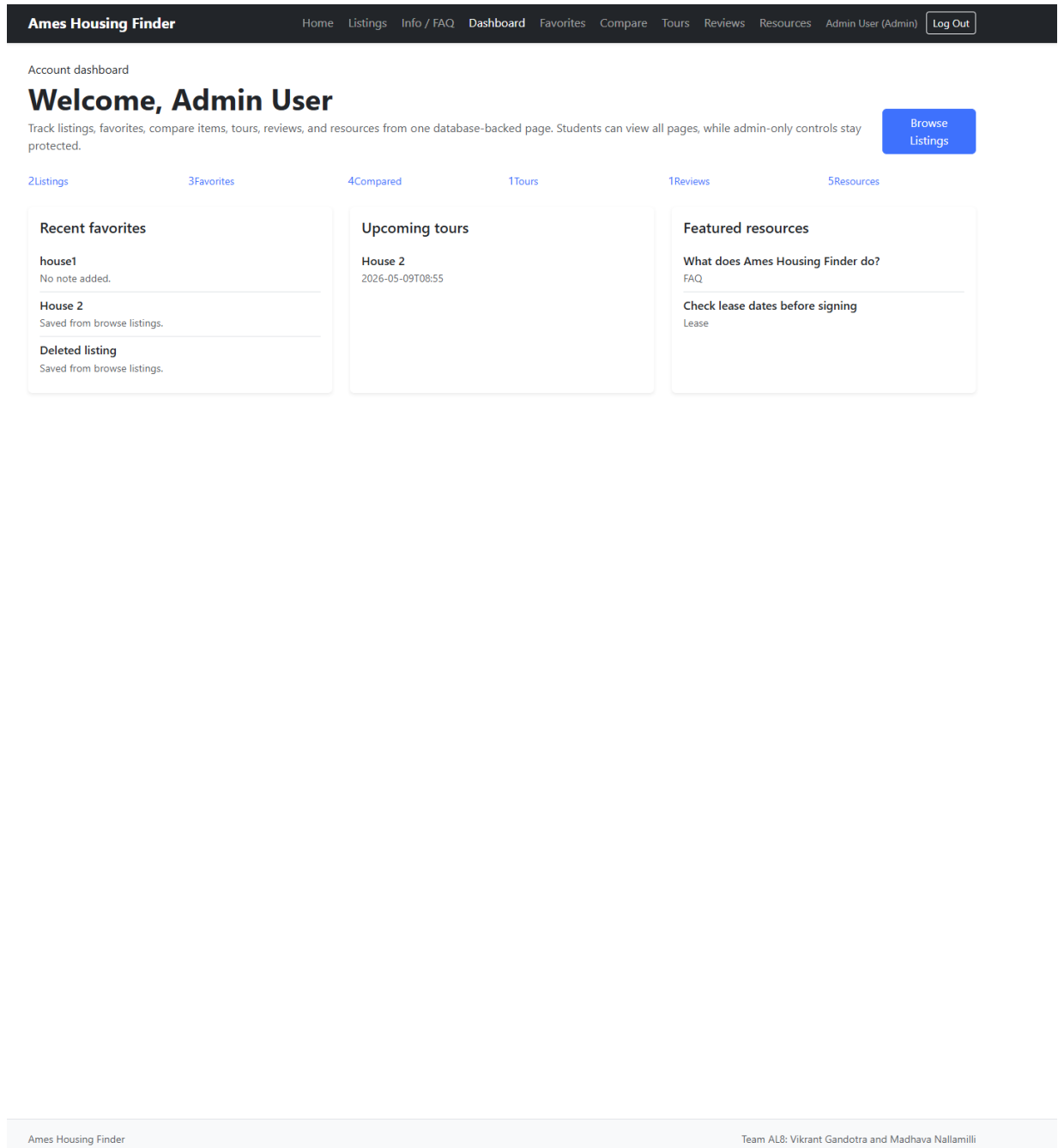


Figure 3: Dashboard page showing account summary counts, recent favorites, upcoming tours, and featured resources. It is protected behind login.

Ames Housing Finder

[Home](#) [Listings](#) [Info / FAQ](#) [Dashboard](#) [Favorites](#) [Compare](#) [Tours](#) [Reviews](#) [Resources](#) [Admin User \(Admin\)](#) [Log Out](#)

Full CRUD feature

# Favorites

Save, update, prioritize, and remove your saved housing listings.

### Save a listing

| Listing                   | Label                 | Priority          | Note        |
|---------------------------|-----------------------|-------------------|-------------|
| <div>Choose listing</div> | <div>Top choice</div> | <div>Medium</div> | <div></div> |

Save Listing

No listings are currently saved in your favorites.

Ames Housing Finder

Team AL8: Vikrant Gandotra and Madhava Nallamilli

Figure 4: Favorites page. A user chooses a listing, sets a label, priority, and note, then saves the listing to personal favorites.

17

Ames Housing Finder

[Home](#) [Listings](#) [Info / FAQ](#) [Dashboard](#) [Favorites](#) [Compare](#) [Tours](#) [Reviews](#) [Resources](#) [Admin User \(Admin\)](#) [Log Out](#)

Full CRUD feature

# Compare Listings

Add up to three listings, update comparison notes, and remove listings from your compare list.

Add listing to compare

Listing

Choose listing

Compare Note

Why compare this option?

Add to Compare

No listings are currently in your compare list.

Ames Housing Finder

Team AL8: Vikrant Gandotra and Madhava Nallamilli

Figure 5: Compare Listings page. Users add listings to a compare list and write comparison notes. The backend enforces a maximum of three selections.

18

Ames Housing Finder

[Home](#) [Listings](#) [Info / FAQ](#) [Dashboard](#) [Favorites](#) [Compare](#) [Tours](#) [Reviews](#) [Resources](#) [Admin User \(Admin\)](#) [Log Out](#)

Full CRUD feature

# Tour Appointments

Create, update, confirm, cancel, and delete housing tour requests.

Schedule a tour

Listing

Choose listing

▼

Name

Admin User

Email

admin@iastate.edu

Status

Requested

▼

Tour Date

mm/dd/yyyy --:-- --

📅

Note

Request Tour

House 2

2026-05-09T08:55

Requested

Visitor: jose

Email: jose@gmail.com

User ID: 6a0119c3e579b9880b03fa71

No note added.

Edit

Confirm

Cancel

Delete

Ames Housing Finder

Team AL8: Vikrant Gandotra and Madhava Nallamilli

Figure 6: Tour Appointments page. Users create tour requests and can edit, confirm, cancel, or delete appointments.

Ames Housing Finder

[Home](#) [Listings](#) [Info / FAQ](#) [Dashboard](#) [Favorites](#) [Compare](#) [Tours](#) [Reviews](#) [Resources](#) [Admin User \(Admin\)](#) [Log Out](#)

Student reviews and admin moderation

# Reviews

Students can create, edit, and delete their own reviews. Admins can delete reviews for moderation, but cannot edit reviews written by other users.

Create review

Listing

Choose listing

Rating

5

Author

Admin User

Title

Comment

Create Review

This House sucks

house1

Author: jose

good

Delete

3/5

Ames Housing Finder

Team ALB: Vikrant Gandotra and Madhava Nallamilli

Figure 7: Reviews page. Students can create reviews; admins can delete reviews for moderation but cannot edit another user’s review.

Ames Housing Finder

[Home](#) [Listings](#) [Info / FAQ](#) [Dashboard](#) [Favorites](#) [Compare](#) [Tours](#) [Reviews](#) [Resources](#) [Admin User \(Admin\)](#) [Log Out](#)

Admin-managed support content

# Housing Resources

Students can view resources. Admins can create, edit, and delete housing resource records.

Create resource

Category

FAQ

▼

Title

Link

Body

☐ Feature on dashboard

Create Resource

All

FAQ

Budget

Lease

Moving

Campus

FAQ

Featured

What does Ames Housing Finder do?

It helps students browse listings, save favorites, compare choices, schedule tours, and review housing options.

Edit

Delete

Lease

Featured

Check lease dates before signing

Students should confirm move-in date, renewal rules, subleasing rules, pet policies, and maintenance expectations.

Edit

Delete

Budget

Budget

Budget

[Open resource](#)

Edit

Delete

Campus

Think about CyRide access

Transit access can matter as much as distance, especially during winter or early morning classes.

Edit

Delete

Moving

Plan move-in early

Before moving, confirm keys, parking permits, utilities, and roommate responsibilities.

Edit

Delete

Ames Housing Finder

Team AL8: Vikrant Gandotra and Madhava Nallamilli

Figure 8: Resources page. Admins can create, edit, delete, categorize, feature, and filter housing resource records.

Ames Housing Finder

[Home](#) [Listings](#) [Info / FAQ](#) [Login](#) [Sign Up](#)

Required page

### Log in

Log in to use favorites, compare, tours, resources, dashboard tools, and admin-only management.

Email

Password

Log In

Need an account? [Create one.](#)

Ames Housing Finder

Team AL8: Vikrant Gandotra and Madhava Nallamilli

Figure 9: Login page. Users submit email and password to the auth route and receive a stored user session on success.

Ames Housing Finder

[Home](#) [Listings](#) [Info / FAQ](#) [Login](#) [Sign Up](#)

Required page

### Create a student account

Public signup creates a student account. Admin accounts are seeded by the backend and cannot be created from this form.

Name

Email

Password

Sign Up

Already have an account? [Log in](#).

Ames Housing Finder

Team AL8: Vikrant Gandotra and Madhava Nallamilli

Figure 10: Signup page. Public signup creates a Student account; admin accounts are seeded or created directly in the database.

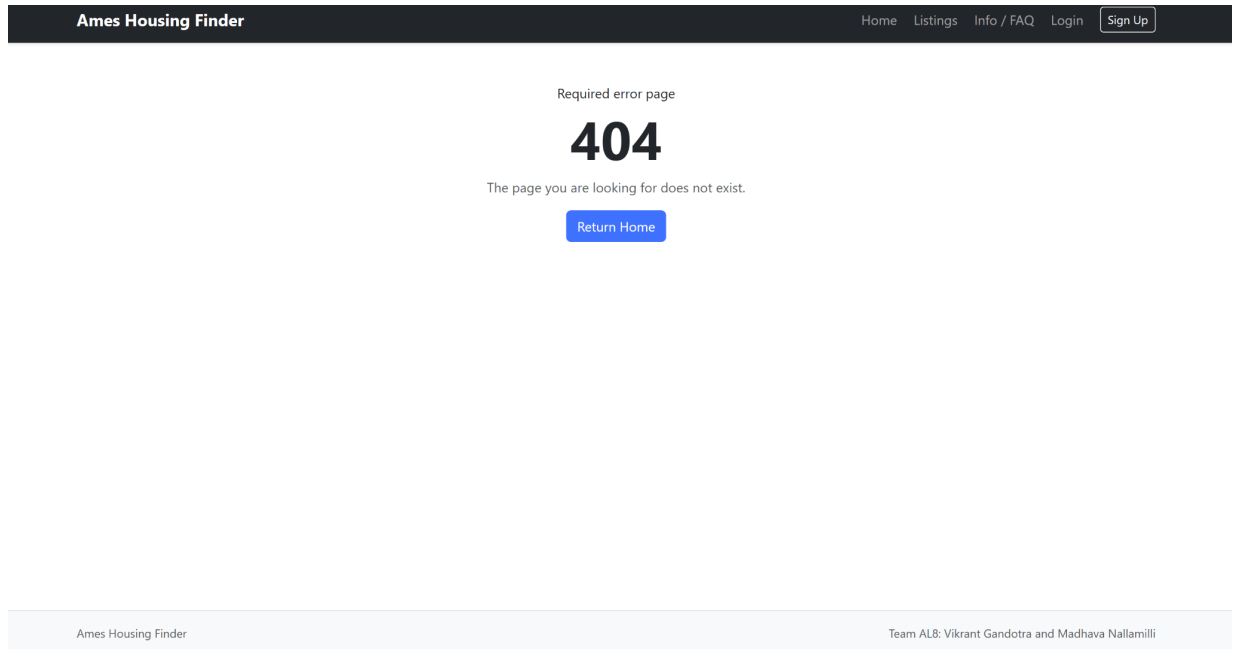


Figure 11: 404 page. Unknown frontend routes are handled by a friendly error page with a Return Home action.

## 6 Installation and Setup Instructions

### 6.1 Frontend Setup

The frontend requires Node.js and npm. From the project root, open a terminal and run:

```
cd frontend
npm install
npm run dev
```

The Vite development server prints a local URL, usually `http://localhost:5173`. The frontend uses Axios with the base URL `http://localhost:5000/api`, so the backend must be running on port 5000 unless `frontend/src/services/api.js` is changed.

### 6.2 Backend Setup

The backend also requires Node.js and npm. From the project root, open a second terminal and run:

```
cd backend
npm install
cp .env.example .env
npm run seed
npm run dev
```

The backend package includes Express, CORS, dotenv, Mongoose, and nodemon. The `npmrun dev` command starts `nodemonserver.js`; `npmstart` starts `nodeserver.js`.

## 6.3 Database Setup

The project uses MongoDB. For local development, MongoDB should run at:

```
mongodb://127.0.0.1:27017/amesHousingFinder
```

The backend reads the database connection string from `backend/.env`. The seed script can be run with `npmrun seed` to create demo data. The seeded demo accounts are:

- Student: `gandotra@iastate.edu`, password `password123`
- Admin: `admin@iastate.edu`, password `password123`

## 6.4 Environment Variables

The backend uses the following environment variables:

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/amesHousingFinder
```

The project does not require external API keys. If deployed to a different server, the frontend Axios base URL and backend CORS configuration should be updated to match the deployed domains.

## 7 Contribution Overview

### 7.1 Vikrant Gandotra Contributions

Vikrant worked on the public-facing listing experience along with their CRUD, general application structure, navigation, styling, and the favorite page and its CRUD along with all the backend routes connected to these files.

Specific responsibilities included:

- Built listing browsing, filtering, sorting, and details-page behavior.
- Implemented admin-only listing create, update, and delete controls.
- Built the listing browsing workflow, including listing card display, listing detail pages, search/filter/sort UI, and admin listing management behavior.
- Worked on `Listing.js`, `listings.js`, `Listings.jsx`, `ListingDetails.jsx`, and `ListingCard.jsx` so listings could be created, read, updated, deleted, searched, and filtered.
- Built the Favorites module with the `Favorite` model, favorites routes, and `Favorites.jsx`, including label, priority, note, and delete/update behavior.
- Implemented the shared application shell, including `App.jsx`, navigation behavior, protected routes, footer, home page, dashboard UI, 404 page, and global CSS styling.
- Helped test the user workflows through the browser and verified that guest, student, and admin navigation showed the correct pages and controls.

### 7.2 Madhava Nallamilli Contributions

Madhava worked on the code routes and pages identified for Compare CRUD, Tour Appointments CRUD, Auth/Login/Signup, Reviews, Resources/FAQ, and backend startup/-dashboard/seed files.

- Implemented Compare CRUD using `CompareSelection.js`, `compare.js`, and `Compare.jsx`, including duplicate prevention and the three-listing compare limit.

- Implemented Tour Appointments CRUD using `TourAppointment.js`, `appointments.js`, and `Appointments.jsx`, including requested, confirmed, completed, and canceled statuses.
- Implemented authentication using `User.js`, `auth.js`, `Login.jsx`, and `Signup.jsx`, including student signup and seeded admin login behavior.
- Implemented Reviews using `ListingReview.js`, `reviews.js`, and `Reviews.jsx`, including role rules that allow admins to delete but not edit reviews from other users.
- Implemented Resources and FAQ using `HousingResource.js`, `resources.js`, `Resources.jsx`, and `FAQ.jsx`, including categories, featured resources, and dashboard/FAQ display.
- Worked on `backend/server.js`, `backend/api/routes/dashboard.js`, and `backend/api/seed.js` to mount routes, summarize data, and seed demo records.

## 8 Challenges Faced

### 8.1 Vikrant Gandotra Challenges

One challenge was making the Listings page useful without making the UI too complicated. The listing route supports many query parameters, so the frontend had to keep form state, dropdown state, and sorting state synchronized with the backend results. This was diagnosed by testing each filter one at a time and verifying the returned records in the browser. The solution was to centralize the search/filter inputs and send them as query parameters to `/api/listings`.

Another challenge was role-based interface behavior. Admin users needed create/edit/delete controls, but guests and students should not see admin-only listing controls. This required keeping the current user available in the application shell and passing it into the pages that needed it. The solution was to use the session service and role helper functions so that UI decisions were based on a normalized stored user.

A third challenge was keeping the page layout consistent across many pages. Because the app includes forms, cards, dashboards, and error pages, inconsistent spacing or button styling could make the project feel unfinished. The solution was to use Bootstrap classes together with shared global CSS and reusable components such as the navbar, footer, and listing cards.

## 8.2 Madhava Nallamilli Challenges

One challenge was enforcing personal-data rules on Compare, Favorites, Tours, and Reviews. The backend needed to know not only which user record was being requested, but also which user was acting. This was diagnosed by testing requests as student and admin users. The solution was to send `userId` and `actorId`, load the actor from MongoDB, and check ownership or admin role before allowing updates and deletes.

Another challenge was preventing duplicate or invalid compare records. A user should not be able to add the same listing twice, add a listing that does not exist, or compare more than three listings at once. The solution used an index on `userId` and `listingId`, explicit validation in the POST route, `Listing.findById()`, and `countDocuments()` before creating a record.

A third challenge was making the Resources and FAQ pages share data without duplicating content. FAQ entries needed to appear on the public FAQ page while resources also needed admin CRUD controls and dashboard visibility. The solution was to use a single `HousingResource` model with categories and an `isFeatured` flag. The Resources page manages the records, while FAQ and Dashboard read filtered subsets.

## 9 Final Reflections

### 9.1 Vikrant Gandotra Reflection

This project improved my confidence with React page structure, Bootstrap styling, React Router navigation, and full-stack debugging. The most rewarding part was seeing the listing search and browsing experience work with real MongoDB data instead of static placeholder objects. If starting over, I would design a more formal shared form component system earlier so that all CRUD pages reuse the same validation and button patterns. The project strengthened skills in frontend organization, API integration, and designing interfaces around user roles.

## 9.2 Madhava Nallamilli Reflection

This project improved my confidence with Express routes, Mongoose schemas, MongoDB validation, and connecting CRUD pages to backend endpoints. The most rewarding part was building several complete end-to-end features where a form submission immediately changed database-backed records and the UI refreshed to show the result. If starting over, I would add a stronger authentication layer with hashed passwords and token-based authorization instead of passing actor ids from the frontend. The project helped build practical experience with route design, database relationships, error handling, and role-aware user workflows.